

Competitive Programming

Section 42: Advanced Data Structures Mixed Contest

Competitive Programming Course

2026-07-10

Table of contents

1	Choosing the right query structure	3
1.1	Dynamic order statistics	3
1.2	Range add and range sum with two Fenwick trees	3
1.3	Static rectangle counting	4
1.4	Recognition table	4
1.5	Full rectangle-count example	4
1.6	Fenwick k-th implementation detail	5
2	Exercises	6
3	Worked update traces	7
3.1	Dynamic order statistics	7
3.2	Two Fenwick trees for range add/range sum	7
3.3	Static rectangle count: inclusion-exclusion in x	7
4	Proof habits, language notes, and workflow	8

1 Choosing the right query structure

This contest block mixes three common advanced query reductions: ranks with Fenwick binary lifting, range-add/range-sum with two Fenwick trees, and offline two-dimensional counting with coordinate compression. The common habit is to rewrite every query as a prefix query.

1.1 Dynamic order statistics

A multiset over values $1..m$ can be represented by a frequency array:

```
freq[x] = how many copies of value x are currently present
```

A Fenwick tree over `freq` supports:

- insert `x`: add +1 at index `x`;
- erase one `x`: add -1 at index `x`;
- count values $\leq x$: prefix sum `sum(x)`;
- find the `k`-th smallest: find the first position whose prefix sum is at least `k`.

The `k`-th operation is Fenwick binary lifting. Instead of binary searching and calling `sum(mid)` repeatedly, we walk down powers of two inside the Fenwick tree:

```
pos = 0
for step = largest power of two down to 1:
    if pos + step <= n and bit[pos + step] < k:
        k -= bit[pos + step]
        pos += step
answer = pos + 1
```

The invariant is: after considering a step, all indices up to `pos` are known to have prefix sum strictly less than the original `k`.

1.2 Range add and range sum with two Fenwick trees

One Fenwick tree can store a difference array and answer point queries after range updates. For range-sum queries after range updates, store two trees.

If we add `x` to every position in $[1, r]$, update:

```
B1[1] += x, B1[r+1] -= x
B2[1] += x*(1-1), B2[r+1] -= x*r
```

Then the prefix sum of the actual array up to `p` is:

```
prefix(p) = p * sum(B1, p) - sum(B2, p)
```

So:

```
range_sum(l, r) = prefix(r) - prefix(l-1)
```

This formula is just summing a difference array algebraically. B1 controls the slope of the prefix sum, and B2 cancels the extra offset introduced when an update starts later than position 1.

1.3 Static rectangle counting

For points (x, y) and rectangle queries $[x1, x2] * [y1, y2]$, reduce each rectangle to four prefix queries:

```
count(x <= X, y <= Y)
```

Then:

```
rect = F(x2,y2) - F(x1-1,y2) - F(x2,y1-1) + F(x1-1,y1-1)
```

Sort points by x . Sort prefix queries by X . Sweep from left to right and activate each point's compressed y coordinate in a Fenwick tree. Each prefix query becomes one Fenwick prefix sum over y .

This is the same offline pattern as Section 13: sort by the threshold, add objects that now qualify, and answer each query from a dynamic prefix data structure.

1.4 Recognition table

Statement wording	Model
"k-th smallest under updates"	Fenwick frequencies plus binary lifting
"add on interval, sum on interval"	two Fenwick trees
"static points inside many rectangles"	offline sweep plus compressed Fenwick
"arbitrary online min/max on intervals"	segment tree

1.5 Full rectangle-count example

Points:

```
(1,2), (3,1), (4,4), (6,2)
```

Query rectangle:

```
2 <= x <= 5, 1 <= y <= 3
```

Use four prefix dominance counts:

```
F(5,3) - F(1,3) - F(5,0) + F(1,0)
```

$F(5,3)$ counts points with $x \leq 5$ and $y \leq 3$: $(1,2)$, $(3,1)$. $F(1,3)$ removes points left of the rectangle: $(1,2)$. The other two terms are zero. Final answer is 1, namely $(3,1)$.

This example is worth doing by hand because most bugs in offline rectangle queries are sign mistakes in the four-prefix formula.

1.6 Fenwick k-th implementation detail

The binary lifting search assumes all frequencies are nonnegative. That is true for multisets and alive-position arrays, but not for arbitrary prefix sums.

Pseudo-code:

```
pos = 0
for step from highest_power down to 1:
    if pos+step <= n and bit[pos+step] < k:
        k -= bit[pos+step]
        pos += step
return pos+1
```

This searches directly inside the Fenwick representation. It is faster than external binary search and avoids an extra logarithmic factor.

2 Exercises

3 Worked update traces

3.1 Dynamic order statistics

The Fenwick tree stores multiplicities, not values. With inserts 3,5,3, its prefix counts are $\text{count}(\leq 1)=0$, $\text{count}(\leq 3)=2$, $\text{count}(\leq 5)=3$. The second element is found by searching the first index whose prefix count is at least two, namely three. After erasing one three, the second element becomes five.

For $\text{kth}(k)$, keep a tentative index whose prefix sum is strictly less than the remaining rank. Try Fenwick powers of two from largest to smallest; accept a step only when its stored partial sum is still below that rank. The next index is exactly the first prefix reaching k . Reject k outside $1..total_count$ before this search.

3.2 Two Fenwick trees for range add/range sum

Adding x on $[1,r]$ changes the difference array by $+x$ at 1 and $-x$ at $r+1$. Let B1 store those changes and B2 store each change times its index correction. Then the prefix sum of the reconstructed array is

```
prefix(i) = i * sum(B1,i) - sum(B2,i).
```

For an add of ten on $[2,4]$, record $B1[2] += 10, B1[5] -= 10$ and $B2[2] += 10, B2[5] -= 40$. At $i=3$, the formula adds $3*10-10=20$, the two added positions in the prefix. A range sum is $\text{prefix}(r)-\text{prefix}(l-1)$.

3.3 Static rectangle count: inclusion-exclusion in x

For each rectangle $[x1,x2] \times [y1,y2]$, count points with $x \leq x2$ and y in its interval, then subtract the same count at $x1-1$. Sort point insertions and these two events by x . A compressed- y Fenwick tree contains exactly points with x no larger than the current event's x . For points $(1,1), (2,3), (4,2)$ and rectangle $[1,3] \times [1,3]$, the $x=3$ event sees two points and $x=0$ sees zero.

4 Proof habits, language notes, and workflow

All three structures are summaries with explicit meanings: Fenwick prefix sum is a multiplicity/count, the pair of trees reconstructs a prefix of values, and the sweep tree represents points already passed by x . Prove updates preserve that meaning before trusting a query formula.

C++. Use `long long` for range values, Fenwick internal sums, and answers; the product $x*(1-1)$ can overflow `int`. Compression indices are one-based.

Python. Iterative list-based Fenwick trees are viable for the local limits. Coordinate-compress all y values from points and query boundaries before a sweep; do not use a dictionary as a substitute for order statistics.

Problem	Essential adversarial cases
A	duplicate insert/erase, invalid k , <code>count</code> $\leq x$ at boundaries
B	negative add, single-cell range, full range, $l=1$
C	point on every rectangle boundary, duplicate y , empty rectangle

Run `CP_TARGET=solution python3 sections/42_advanced_data_structures_mixed/check.py`. For small randomized inputs, compare A to a sorted list, B to an array updated directly, and C to a direct scan of every point for every rectangle.

- A. Dynamic Order Statistics: Maintain a multiset over values $1..m$: insert, erase one copy, k -th, and `count` $\leq x$.
- B. Range Add Range Sum: Support range addition and range-sum queries.
- C. Static Rectangle Count: Given points and rectangle queries, count points inside each rectangle.